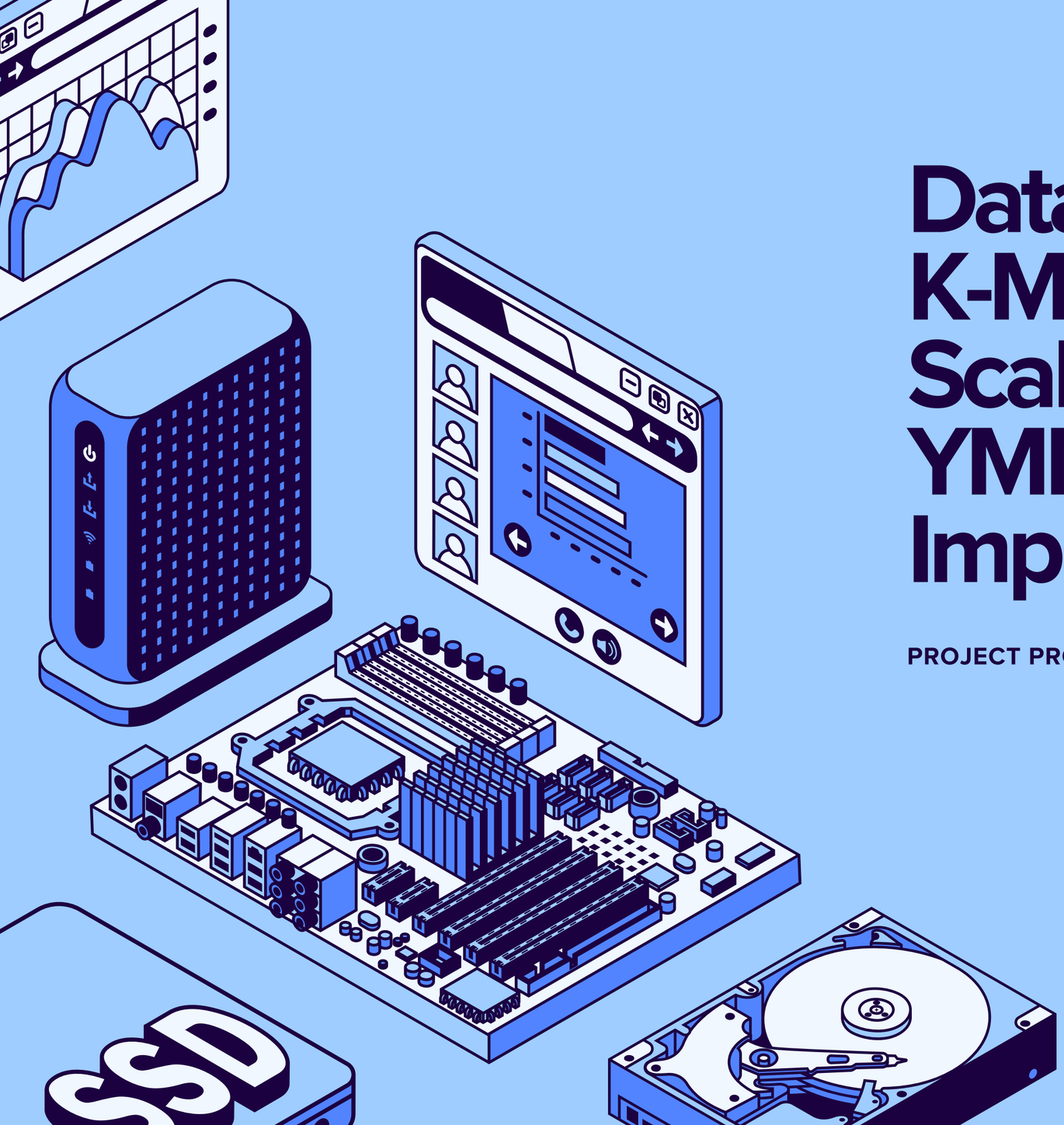


Data-Level Parallelism in K-Means Clustering: Scalar C versus XMM and YMM SIMD Implementations

PROJECT PROPOSAL

Daniel Jedrick C. Dabuit
Ethan Daniel B. Magbatoc
Jose Bryan L. Perez
Tara Ysabel P. Uy

S01 CEPARCO Group 2

Abstract

This project evaluates how much data-level parallelism improves the performance of k-means clustering. A sequential **C implementation** serves as the baseline and is compared against two SIMD implementations written in x86-64 assembly: one using **XMM** registers and another using **YMM** registers. The parallelization targets the assignment step of Lloyd's algorithm, which computes the distance between every data point and every cluster centroid. The three versions are compared on **execution time, speedup, and accuracy** across several numbers of points (N) and clusters (K). The goal is to show how SIMD register width affects k-means performance and where the speedup stops.

Inputs

Input	Description
X[], Y[]	Single-precision (float32) coordinates of the N points, stored as two separate arrays (structure-of-arrays)
N	Number of data points
K	Number of clusters
MAX_ITER	Maximum number of iterations
TOL	Convergence threshold on centroid movement
SEED	Fixed random seed, so all versions receive identical data and identical initial centroids

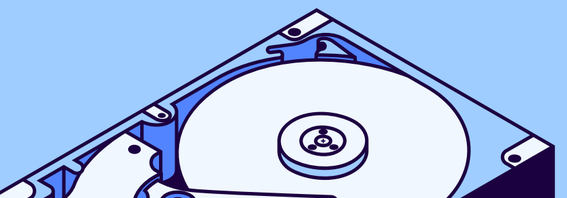
Parallelized Processes

Assignment Step

Squared distance from each point to every centroid; keep the smallest. Points are independent, so it is data-parallel: 4 points per instruction (XMM), 8 (YMM).

Centroid Accumulation

Per-cluster sums of x and y, and member counts. Vectorized only if time permits, since many points add into the same cluster total.



Related Works

1. Lloyd’s algorithm is the standard formulation of k-means.

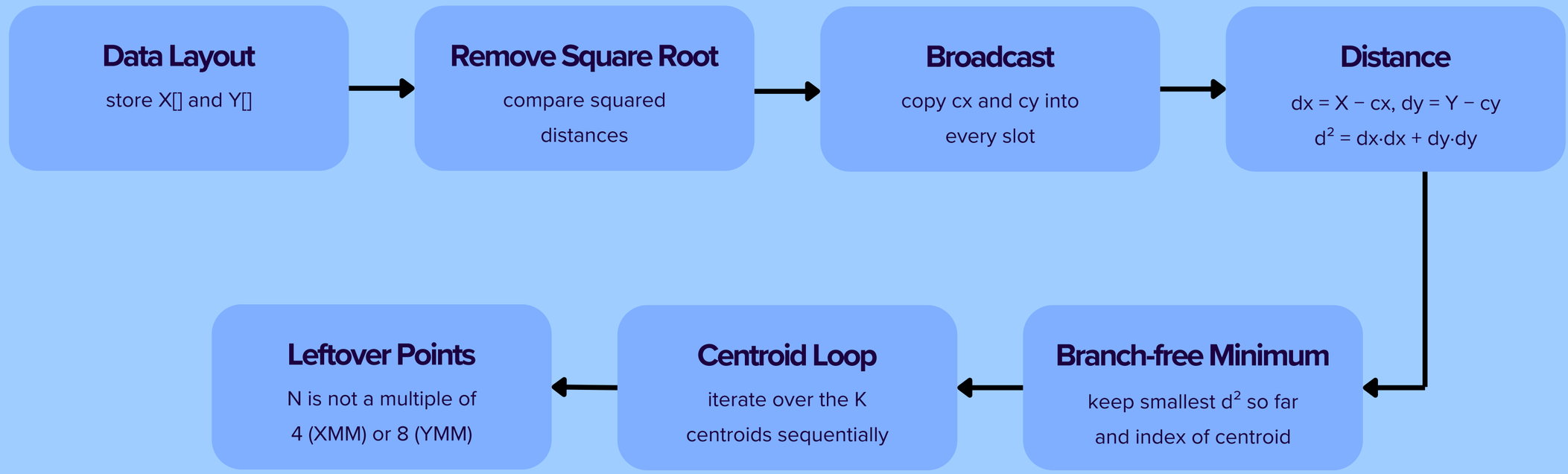
2. FAISS is a library for similarity search and clustering of dense vectors, and its paper notes that GPUs are well suited to data-parallel tasks such as distance computation.

3. NVIDIA's cuVS library provides a GPU-accelerated k-means that follows the same two steps.

ours: hand-written SSE/AVX2 assembly vs. scalar C



Plan for parallelizing the process



Outputs

Final labels of every point, the number of members in each cluster, and the final centroids

Execution time per stage for each version: initial centroid selection, assignment with member counting, centroid recomputation, and full runtime

Speedup of the XMM and YMM versions over C, summarized by the **geometric mean** across the tested sizes

Similarity score: the percentage of points assigned the same label as in the C version, together with the largest centroid difference and the inertia



Proposed Target Implementation Platform

An x86-64 CPU supporting **SSE and AVX2**, running Windows. The driver, baseline, data generation, correctness check, and timing are written in C and built in Visual Studio. The SIMD kernels are written in x86-64 assembly **(NASM)** and linked with the C program. Instruction details are checked against the Intel Intrinsics Guide and the Intel 64 and IA-32 Architectures Software Developer's Manual.

Mid-Project Milestones

Milestone #2 (Week 10): mid-project update demo

- Data generator and timing code completed
- Sequential C k-means completed and verified on a small hand-worked example
- XMM assignment kernel working and producing the same labels as the C version
- YMM kernel started, with preliminary C versus XMM timing results

Milestone #3 (Week 13): complete submission and demo

- YMM assignment kernel completed and verified
- Boundary sizes tested
- Full benchmarks across all N and K in Debug and Release
- Optional vectorized centroid update and optional CUDA version
- GitHub repository with source code, discussion of the parallel algorithms, and the sequential versus parallel execution time comparison
- Optional conference paper

References

- S. P. Lloyd, "Least squares quantization in PCM," IEEE Trans. Information Theory, vol. 28, no. 2, pp. 129-137, 1982. https://www.stat.cmu.edu/~rnugent/PCMI2016/papers/LloydKMeans.pdf

- J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, 2021. https://arxiv.org/abs/1702.08734

- NVIDIA cuVS, "K-Means" clustering guide. https://docs.nvidia.com/cuvs/user-guide/api-guides/clustering-guide/k-means

- Intel, Intel Intrinsics Guide. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

- Intel, Intel 64 and IA-32 Architectures Software Developer’s Manual. https://www.intel.com/content/www/us/en/content-details/825743/intel-64-and-ia-32-architectures-software-developer-s-manual-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4.html

Thank You